



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Всех приветствую!

Как специалисты по тестированию на проникновение, мы всегда пытаемся найти эффективные способы минимизации времени, которое мы тратим на анализ данных. Допустим, нам нужно просканировать подсеть для выявления веб-серверов и определения того, представляют ли они интерес для проведения атак. Сканирование портов – это процесс проверки TCP или UDP портов на удаленной машине с целью обнаружения того, какие службы запущены на определенной цели и потенциально какие векторы атак существуют. Мы обсудим сканирование портов более подробно в другом модуле, а пока давайте оставим это в общих чертах, поскольку это отличный пример, который показывает, как Bash-сценарии могут автоматизировать довольно утомительную задачу.

Мы сначала просканируем порты всей нашей подсети, чтобы определить запущенные службы.

Для начала давайте создадим временную папку, которая будет использоваться для этого упражнения:

```
kali@kali:~$ mkdir temp
```

```
kali@kali:~$ cd temp
```

Теперь, когда мы создали каталог и вошли в него с помощью cd, давайте перейдем к более интересной части - сканированию подсети. Мы будем использовать nmap (который мы обсудим в следующем модуле) в качестве нашего инструмента для сканирования портов:

```
kali@kali:~/temp$ sudo nmap -A --open 192.168.0.1/24 -oG nmap-scan
```

```
(kali@kali)-[~/temp]
└─$ sudo nmap -A --open 192.168.0.1/24 -oG nmap-scan

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

#1) Respect the privacy of others.
#2) Think before you type.
#3) With great power comes great responsibility.

[sudo] password for kali:
Starting Nmap 7.91 ( https://nmap.org ) at 2022-01-30 10:05 EST
Nmap scan report for dlinkrouter.Dlink (192.168.0.1)
Host is up (0.00079s latency).
Not shown: 993 closed ports
PORT      STATE SERVICE        VERSION
53/tcp    open  domain         dnsmasq 2.78
|_ dns-nsid:
|_   bind.version: dnsmasq-2.78
80/tcp    open  http
|_ fingerprint-strings:
|_   FourOhFourRequest:
|_     HTTP/1.1 200 OK
|_     Content-Security-Policy: default-src *; style-src 'self' 'unsafe-in
; script-src 'self' 'unsafe-inline' 'unsafe-eval'; frame-ancestors 'self
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Date: Sun, 30 Jan 2022 15:05:42 GMT
Etag: "61b0bb19.1676"
Content-Type: text/html
Content-Length: 1676
Connection: close
Accept-Ranges: bytes
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "htt
www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"><html xmlns="http://www
org/1999/xhtml" ng-app="error404" ng-controller="Error404Ctrl"><head><tit
g-bind="error404_title" | translate"></title><meta http-equiv="Content-T
content="text/html; charset=utf-8"><meta http-equiv="X-UA-Compatible" co
t="IE=edge"><meta http-equiv="pragma" content="no-cache"><meta http-equiv
che-control" content="no-cache"><me
GenericLines:
HTTP/1.1 500 Server Error
Content-Length: 42
Date: Sun, 30 Jan 2022 15:05:42 GMT
Connection: close
Error 500: Server Error
request: [
GetRequest, HTTPOptions:
HTTP/1.1 303 See other
Location: /admin/index.html
```

Это довольно простое сканирование, с параметром -A для агрессивного сканирования, --open для возврата машин только с открытыми портами, и -oG для сохранения результатов сканирования. Опять же, не волнуйтесь, если nmap для вас в новинку. Мы рассмотрим подробности позже.

Давайте посмотрим на выходной файл, чтобы ознакомиться с его форматом:

```
kali@kali:~/temp$ cat nmap-scan
```

```
(kali@kali)-[~/temp]
└─$ cat nmap-scan
# Nmap 7.91 scan initiated Sun Jan 30 10:05:25 2022 as: nmap -A --open -oG nmap-scan 192.168.0.1/24
Host: 192.168.0.1 (dlinkrouter.Dlink) Status: Up
Host: 192.168.0.1 (dlinkrouter.Dlink) Ports: 53/open/tcp//domain//dnsmasq 2.78/, 80/open/tcp//http//, 81/open/tcp//hosts2-ns?//, 443/open/tcp//ssl|https//, 4445/open/tcp//ssl|upnotifyp?//, 8888/open/tcp//sun-answerbook?//, 9999/open/tcp//abyss?// Ignored State: closed (993) Seq Index: 258 IP ID Seq: All zeros
Host: 192.168.0.140 (Apple-TV.Dlink) Status: Up
Host: 192.168.0.140 (Apple-TV.Dlink) Ports: 3689/open/tcp//daap//Apple iTunes DAAP 11.1b37/, 5000/open/tcp//rtsp//AirTunes rtspd 220.68/, 7000/open/tcp//rtsp//AirTunes rtspd 220.68/, 7100/open/tcp//http//Apple AirPlay httpd/, 62078/open/tcp//tcpwrapped// OS: Apple Mac OS X 10.7.0 (Lion) - 10.12 (Sierra) or iOS 4.1 - 9.3.3 (Darwin 10.0.0 - 16.4.0) Seq Index: 263 IP ID Seq: Randomized
Host: 192.168.0.171 () Status: Up
Host: 192.168.0.171 () Ports: 62078/open/tcp//tcpwrapped// Seq Index: 264 IP ID Seq: All zeros
Host: 192.168.0.174 () Status: Up
Host: 192.168.0.174 () Ports: 21/open/tcp//ftp//Microsoft ftpd/, 80/open/tcp//http//Microsoft IIS httpd 10.0/, 135/open/tcp//msrpc//Microsoft Windows RPC/, 139/open/tcp//netbios-ssn//Microsoft Windows netbios-ssn/, 445/open/tcp//microsoft-ds//Windows 10 Pro 15063 microsoft-ds (workgroup: WORKGROUP)/, 1688/open/tcp//msrpc//Microsoft Windows RPC/, 7070/open/tcp//ssl|realserver?// Ignored State: filtered (993) Seq Index: 262 IP ID Seq: Incremental
# Nmap done at Sun Jan 30 10:07:28 2022 -- 256 IP addresses (5 hosts up) scanned in 123.00 seconds
```

Каждый IP-адрес повторяется дважды, первая строка отображает статус машины, а вторая - номер(а) сканируемого порта. Поскольку нас интересуют только уникальные IP-адреса, необходимо произвести некоторую очистку. Давайте выполним grep для строк, содержащих порт 80:

```
kali@kali:~/temp$ cat nmap-scan | grep 80
```

```
(kali@kali)-[~/temp]
$ cat nmap-scan | grep 80
Host: 192.168.0.1 (dlinkrouter.Dlink)  Ports: 53/open/tcp//domain//dnsmasq 2.78/, 80/open/tcp//http//, 81/open/tcp//hosts2-ns?//, 443/open/tcp//ssl|https//, 4445/open/tcp//ssl|upnotifyp?//, 8888/open/tcp//sun-answerbook?//, 9999/open/tcp//abyss?//  Ignored State: closed (993)  Seq Index: 258  IP ID Seq: All zeros
Host: 192.168.0.174 ( )  Ports: 21/open/tcp//ftp//Microsoft ftpd/, 80/open/tcp//http//Microsoft IIS httpd 10.0/, 135/open/tcp//msrpc//Microsoft Windows RPC/, 139/open/tcp//netbios-ssn//Microsoft Windows netbios-ssn/, 445/open/tcp//microsoft-ds//Windows 10 Pro 15063 microsoft-ds (workgroup: WORKGROUP)/, 1688/open/tcp//msrpc//Microsoft Windows RPC/, 7070/open/tcp//ssl|realserver?//  Ignored State: filtered (993)  Seq Index: 262  IP ID Seq: Incremental
```

Наш результат выглядит гораздо лучше. Давайте попробуем извлечь только IP-адреса, поскольку это все, что нас действительно интересует. Для этого воспользуемся awk для вывода второго поля, используя пробел в качестве разделителя:

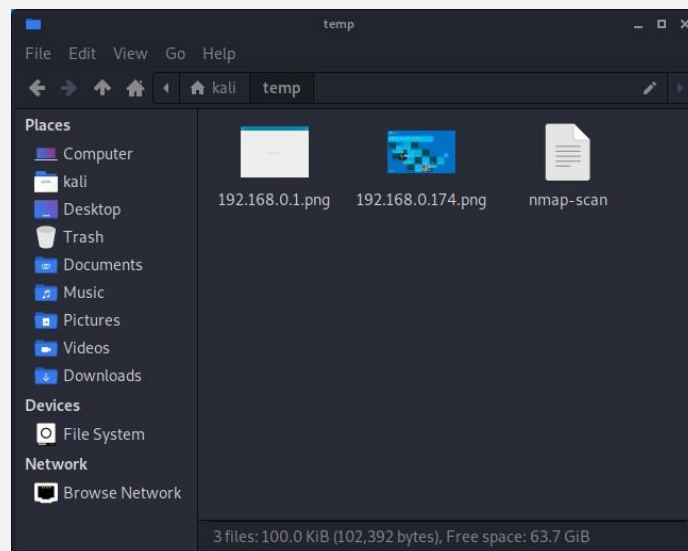
```
kali@kali:~/temp$ cat nmap-scan | grep 80 | awk '{print $2}'
```

```
(kali@kali)-[~/temp]
$ cat nmap-scan | grep 80 | awk '{print $2}'
192.168.0.1
192.168.0.174
```

Хорошо. Теперь у нас есть чистый список IP-адресов. Далее мы воспользуемся однострочной командой Bash, чтобы просмотреть веб-страницы через список IP-адресов. Для этого запустим cutycapt. Мы будем использовать -url, чтобы указать целевой веб-сайт, и -out, чтобы указать имя выходного файла:

```
kali@kali:~/temp$ for ip in $(cat nmap-scan | grep 80 | awk '{print $2}'); do cutycapt --url=$ip --out=$ip.png; done
```

Когда наш цикл завершится, мы можем просмотреть выходные файлы:

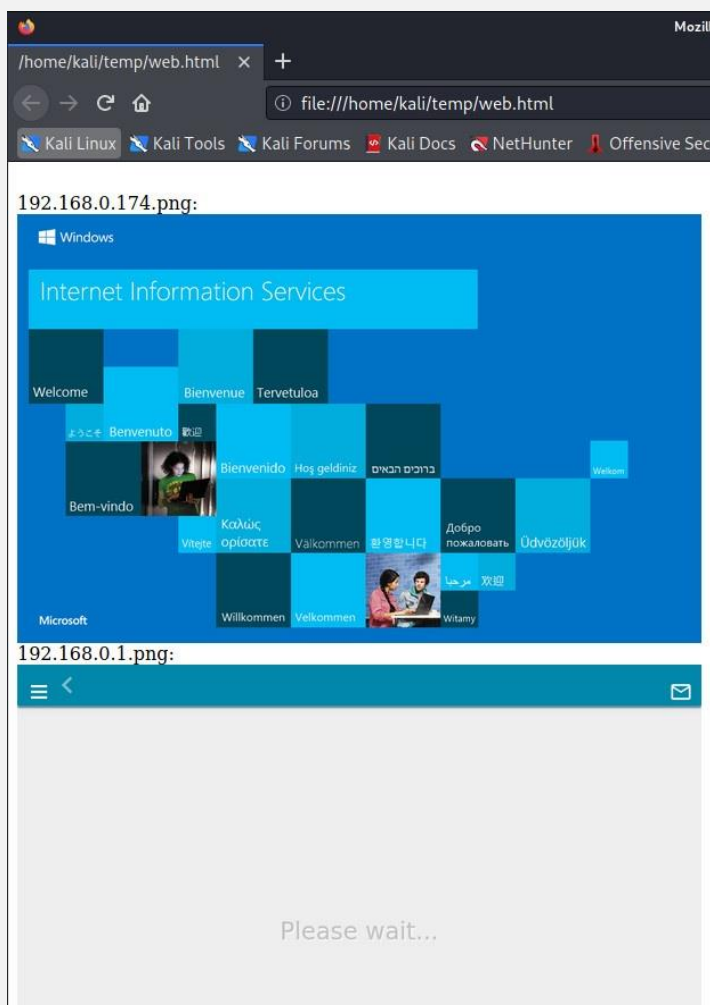


Отлично! Мы приближаемся к нашей цели. Мы могли бы изучить эти файлы по отдельности, но более разумное решение - снова применить наши знания о сценариях и посмотреть, есть ли еще что-то, что мы можем автоматизировать. Для этого потребуются не только навыки работы со сценариями Bash, но и базовые знания HTML:

```
kali@kali:~/temp$ cat > ./pngtohtml.sh
#!/bin/bash
echo "<HTML><BODY><BR>" > web.html
ls -1 *.png | awk -F : '{print $1":\n<BR><IMG SRC=\""$1"\"$2\""
width=600><BR>"}' >> web.html
echo "</BODY></HTML>" >> web.html
kali@kali:~/temp$ chmod +x ./pngtohtml.sh
kali@kali:~/temp$ ./pngtohtml.sh
kali@kali:~/temp$ firefox web.html
```



CODEBU



Этот сценарий создает HTML-файл (web.html), начиная с самых основных тегов (<HTML><BODY>
). Затем ls и awk вставляют каждое название файла .PNG, а затем добавляют имя файла в тег HTML IMG и тем самым добавляют картинки в наш файл web.html. Результат прост, но эффективен: мы получаем представление о главной странице каждого веб-сервера.

Надеюсь, эти практические примеры дали вам представление о некоторых возможностях, которые могут дать сценарии Bash. Эффективное использование Bash будет очень важным при попытке быстрой автоматизации большого количества задач во время тестирования на проникновение.